



**CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY**

An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad-500075, www.cbti.ac.in



COMMITTED TO
RESEARCH,
INNOVATION AND
EDUCATION

47
years

GENERATIVE AI THEORY 22ADE15

ASSIGNMENT-2

REPORT ON

Fine-Tune a Language Model on MHQA

TEAM DETAILS:

BHUKYA JHANSI - 160123748007

BOMMAREDDY SRI SANJANA - 160123748008

BUDAMA LAKSHMI PRAGNAMANASVI - 160123748009

SUBMITTED TO:

Mr. PANIGRAHI SRIKANTH

Assistant Professor

Dept of AIML



ABSTRACT

In the second phase of this project, the Mental Health Question Answering (MHQA) system is extended from a fine-tuned machine learning model into a complete, scalable, and user-interactive application. While the first phase focused on dataset understanding, preprocessing, and model training, this phase emphasizes large-scale implementation, architectural design, deployment strategies, and real-world usability.

The system utilizes a significantly expanded dataset by combining the MHQA and MHQA-b datasets, resulting in approximately 58,617 samples. This large-scale dataset enables improved generalization and robustness of the model across diverse mental health domains such as anxiety, depression, trauma, and obsessive-compulsive disorders. A transformer-based architecture, specifically BERT (bert-base-uncased), is fine-tuned for multiple-choice question answering using supervised learning.

The implementation is carried out using modern deep learning frameworks such as PyTorch and Hugging Face Transformers, with training performed on a GPU-enabled Google Colab environment to ensure computational efficiency. The trained model is then deployed on Hugging Face Hub and integrated into an interactive web application using Gradio.

The developed system allows users to input a question along with multiple answer options and obtain predictions along with confidence scores in real time. The model achieves a validation accuracy of 79.31%, demonstrating its capability to understand and reason over domain-specific mental health queries. This phase highlights the importance of end-to-end system design, including data processing, model training, deployment, and user interface development, in building practical AI solutions.

INTRODUCTION

The evolution of Natural Language Processing (NLP) through transformer-based architectures has enabled machines to perform complex tasks such as question answering, summarization, and contextual reasoning. However, building a functional AI system requires more than just training a model; it involves designing a complete pipeline that includes scalable training, efficient deployment, and intuitive user interaction.

In the domain of mental health, question answering systems play a crucial role in providing accessible and reliable information. Unlike general-purpose question answering, mental health queries often require domain-specific understanding, contextual reasoning, and sensitivity to nuanced language. Therefore, developing an effective system requires both specialized datasets and carefully designed architectures.

In the first phase of this project, a model was fine-tuned using a relatively smaller dataset, and its performance was analyzed. In contrast, the second phase focuses on scaling the system to handle larger datasets, improving model performance, and deploying the model as an interactive application.

This phase introduces several key enhancements. First, a significantly larger dataset is used, combining MHQA and MHQA-b datasets to provide over 58,000 samples. Second, the training pipeline is optimized using GPU acceleration and advanced training strategies. Third, the model is deployed using Hugging Face Hub, enabling easy access and scalability. Finally, a user-friendly interface is developed using Gradio, allowing users to interact with the system in real time.

The primary objective of this phase is to bridge the gap between research and application by transforming a trained model into a fully functional AI system. This demonstrates how machine learning models can be integrated into real-world applications to provide meaningful solutions in sensitive domains such as mental health.

System Architecture

The system architecture of the Mental Health Question Answering (MHQA) application is designed as a modular, layered pipeline that integrates data processing, model training, inference, deployment, and user interaction into a unified framework. The architecture ensures scalability, flexibility, and maintainability, allowing each component to function independently while contributing to the overall system performance.

At a high level, the system follows a structured flow beginning from raw dataset input and ending with user interaction through a web interface. The architecture can be understood as a sequence of interconnected layers, where each layer performs a specific function and passes its output to the next stage. This layered approach enables efficient handling of large datasets, smooth training processes, and real-time inference for user queries.

The first layer of the architecture is the data layer, which consists of the MHQA and MHQA-b datasets. These datasets contain structured multiple-choice questions related to mental health, including domains such as anxiety, depression, trauma, and obsessive-compulsive disorders. The MHQA dataset provides expert-verified samples, while MHQA-b contributes a large volume of pseudo-labeled data. Together, they form a comprehensive dataset of approximately 58,617 samples, enabling the model to learn diverse patterns and improve generalization.

The second layer is the preprocessing layer, which is responsible for cleaning and transforming the raw data into a structured format suitable for training. This layer performs several important operations, including removal of missing values, filtering of valid questions, elimination of duplicate entries, and conversion of labels into a zero-based format. The preprocessing layer ensures that the dataset is consistent, balanced, and free from noise, which is critical for achieving high model performance.

The third layer is the model layer, which consists of the BERT (bert-base-uncased) transformer architecture adapted for multiple-choice question answering. This layer is responsible for understanding the context of the question and evaluating each of the answer options. The model processes the input sequences using self-attention mechanisms, allowing it to capture relationships between words and generate meaningful representations. The multiple-choice classification head assigns scores to each option, enabling the model to select the most appropriate answer.

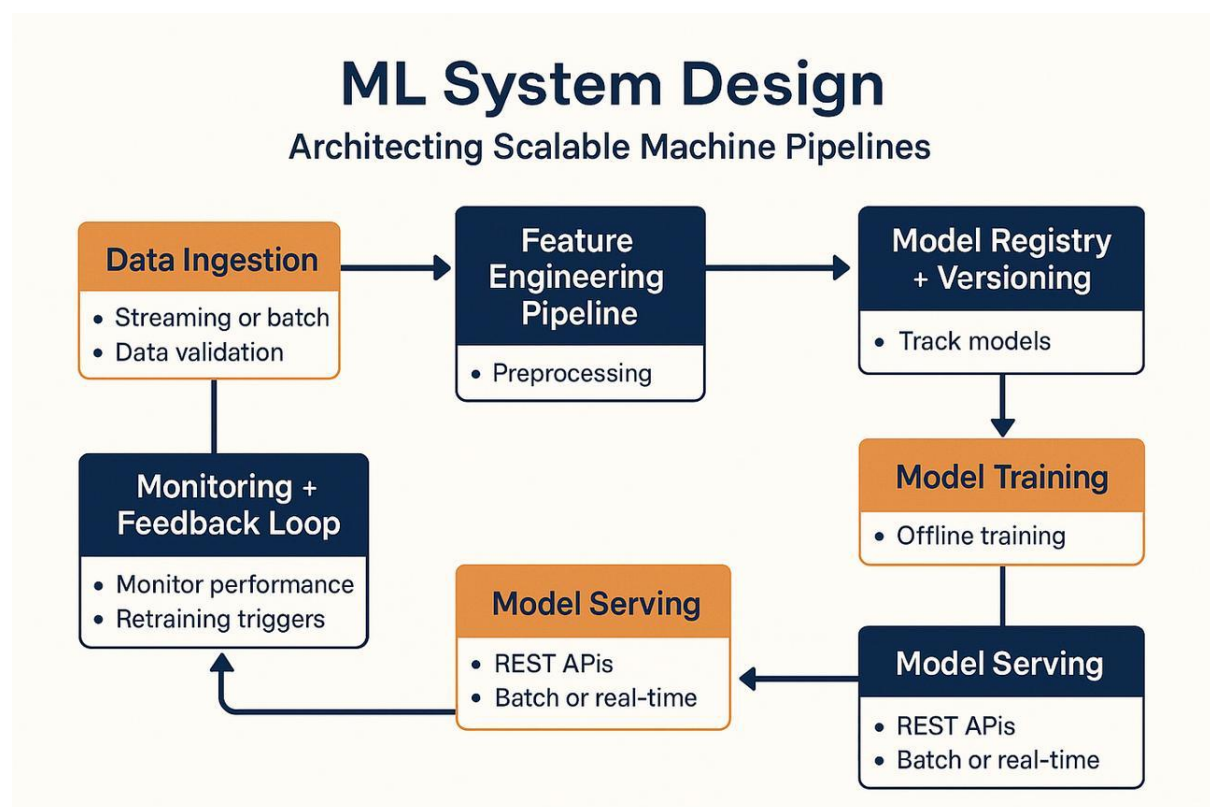
The fourth layer is the training layer, where the model learns from the dataset using supervised learning techniques. This layer includes the training pipeline implemented using the Hugging Face Trainer API. It manages forward propagation, loss computation, backpropagation, and parameter updates. The training process is optimized using techniques such as mixed precision training and weight decay, which

improve efficiency and prevent overfitting. The model is evaluated at regular intervals to monitor performance and ensure convergence.

The fifth layer is the inference layer, which handles real-time prediction. When a user inputs a question and answer options, this layer processes the input using the tokenizer, converts it into numerical format, and passes it through the trained model. The model generates probability scores for each option using the softmax function, and the option with the highest probability is selected as the final prediction. This layer ensures that the system can provide fast and accurate responses.

The sixth layer is the deployment layer, which enables the model to be accessed remotely. The trained model and tokenizer are uploaded to Hugging Face Hub, allowing them to be hosted and shared. The application is deployed using Hugging Face Spaces, which provides an environment for running web-based machine learning applications. This layer ensures scalability and accessibility, allowing users to interact with the system from any device with internet access.

The final layer is the user interface layer, which is built using Gradio. This layer provides a simple and intuitive interface for users to input their questions and view predictions. The interface includes fields for entering the question and answer options, as well as a button to trigger the prediction process. The results are displayed along with confidence scores, providing users with a clear understanding of the model's output.



IMPLEMENTATION

The implementation of the MHQA system involves a comprehensive pipeline that includes environment setup, dataset engineering, preprocessing, model training, evaluation, and deployment. Each stage is carefully designed to ensure efficiency, scalability, and accuracy.

The development environment is set up using Python and a collection of powerful libraries, including PyTorch for deep learning, Hugging Face Transformers for model implementation, Pandas and NumPy for data handling, and Scikit-learn for dataset splitting and evaluation. Google Colab is used as the primary platform for training due to its support for GPU acceleration, specifically the NVIDIA T4 GPU. This significantly reduces training time and allows efficient handling of large datasets.

The dataset used in this phase is constructed by combining the MHQA dataset and the MHQA-b dataset. The MHQA dataset contains approximately 2,475 expert-verified samples, while the MHQA-b dataset contributes over 56,000 pseudo-labeled samples. After combining and preprocessing, the final dataset contains approximately 58,617 samples, providing a rich and diverse dataset for training.

Data preprocessing is a critical step that ensures the quality of the dataset. The preprocessing pipeline involves removing missing values, filtering valid questions, eliminating duplicate entries, and selecting only the relevant columns required for training. The labels are converted from a range of 1–4 to 0–3 to match the expected format of the model. This ensures that the dataset is clean, consistent, and suitable for training.

The dataset is then split into training and validation sets using a stratified sampling approach. Approximately 90% of the data is used for training, while 10% is reserved for validation. Stratified sampling ensures that the distribution of labels remains consistent across both sets, preventing bias and ensuring fair evaluation.

To handle the multiple-choice nature of the task, each question is paired with its four answer options, creating four input sequences per question. These sequences are tokenized using the BERT tokenizer, which converts text into numerical representations such as input IDs, attention masks, and token type IDs. A maximum sequence length of 128 tokens is used to balance computational efficiency and information retention.

The model used in this implementation is BERT (bert-base-uncased), which is adapted for multiple-choice question answering using a classification head. The model processes the input sequences and assigns a score to each answer option. The option with the highest score is selected as the predicted answer.

The training process is managed using the Hugging Face Trainer API, which simplifies the implementation of training and evaluation. The model is trained for three epochs with a batch size of 8. Mixed precision training is enabled to improve computational efficiency, and weight decay is applied to prevent overfitting. The training process involves forward propagation, loss computation using cross-entropy, backpropagation, and parameter updates.

During training, the model's performance is evaluated on the validation set after each epoch. This allows monitoring of accuracy and loss, ensuring that the model is learning effectively. The best model is selected based on validation performance.

Overall, the implementation demonstrates a complete machine learning pipeline, from data preparation to model training and evaluation, providing a strong foundation for deployment and real-world usage.

METHODOLOGY

The methodology followed in this phase is designed to systematically transform the problem of mental health question answering into a machine learning task that can be effectively handled by a transformer-based model. The overall approach is based on supervised learning, where the model is trained on labeled data to predict the correct answer among multiple choices. This methodology ensures that the system is both structured and scalable, enabling it to handle large datasets and complex reasoning tasks effectively.

Problem formulation

The MHQA task is formulated as a multi-class classification problem. Each question is associated with four possible answer options, and the objective of the model is to identify the correct option. Instead of generating answers in free text, the system selects the most appropriate answer from the given choices. This formulation simplifies evaluation and allows the use of standard classification metrics such as accuracy.

This approach also reduces ambiguity in output generation, which is especially important in sensitive domains such as mental health. By restricting the output space to predefined options, the system ensures reliability and consistency in responses. Additionally, this formulation enables efficient training, as the model focuses on learning discriminative features that differentiate correct answers from incorrect ones.

Input representation

A key aspect of the methodology is how the input data is structured. For each question, four separate input pairs are created by combining the question with each answer option. This allows the model to evaluate each option independently while considering the context of the question.

This representation ensures that the model can compare all options simultaneously and assign a probability score to each one. It is particularly effective for multiple-choice tasks, as it leverages the contextual understanding capability of transformer models. By treating each option as a separate input, the model can focus on the semantic relationship between the question and each answer choice.

Moreover, this approach improves the model's ability to handle subtle differences between similar options, which is a common challenge in multiple-choice question answering tasks.

Tokenization strategy

The textual inputs are converted into numerical form using the BERT tokenizer. The tokenizer splits the text into tokens, assigns numerical IDs, and generates attention masks to indicate relevant parts of the input. Tokenization ensures that the model can process textual data efficiently.

A maximum sequence length of 128 tokens is used, which provides a balance between capturing sufficient context and maintaining computational efficiency. Shorter sequences reduce memory usage and training time, while still preserving the essential information required for accurate predictions.

In addition, padding and truncation techniques are applied to ensure uniform input sizes, enabling efficient batch processing during training.

Learning approach

The model is trained using supervised learning, where each input is associated with a correct label. The training objective is to minimize the cross-entropy loss, which measures the difference between predicted probabilities and actual labels.

The optimization process uses the AdamW optimizer, which is well-suited for transformer-based architectures. This optimizer improves convergence and stability by adapting learning rates for different parameters. The learning process involves iterative updates to model parameters through backpropagation, allowing the model to improve its predictions over time.

This approach enables the model to learn complex patterns and relationships within the data, improving its ability to handle diverse question types and domains.

Training strategy

To ensure efficient and stable training, several techniques are applied. Mixed precision training is used to reduce computational cost and improve speed. Weight decay is applied to prevent overfitting by penalizing large weights. Stratified sampling ensures that the dataset remains balanced across classes during training and validation.

The model is trained for multiple epochs, allowing it to learn patterns in the data and improve performance progressively. Early stopping and model checkpointing can also be incorporated to prevent overfitting and retain the best-performing model.

These strategies collectively enhance model performance and ensure efficient utilization of computational resources.

Evaluation strategy

The model is evaluated using validation data after each training epoch. The primary metric used is accuracy, which measures the proportion of correctly predicted

answers. Validation loss is also monitored to assess how well the model generalizes to unseen data.

The best model is selected based on validation performance, ensuring that the final system achieves optimal accuracy. Additional evaluation techniques such as confusion matrices and error analysis can be used to gain deeper insights into model behavior.

FRONTEND DESIGN AND USER INTERFACE

FRONTEND DESIGN AND USER INTERFACE

The frontend of the MHQA system is designed to provide a simple, intuitive, and efficient interface that enables users to interact with the underlying machine learning model without requiring any technical expertise. The interface is developed using Gradio, a lightweight yet powerful framework that allows rapid creation of interactive web applications for machine learning models. By abstracting complex backend operations, Gradio makes it possible to build user-friendly interfaces that directly connect inputs and outputs of the model in real time.

The frontend plays a crucial role in bridging the gap between the trained model and end users. While the model itself performs complex computations, the frontend ensures that these operations are presented in a clear and understandable manner. This is especially important in a domain like mental health, where users may come from diverse backgrounds and may not have prior experience with AI systems.

Design objectives

The primary objective of the frontend design is to ensure usability, accessibility, and clarity. The interface is structured in such a way that even users with no prior knowledge of machine learning can interact with the system effortlessly. The design prioritizes simplicity, ensuring that users are not overwhelmed by unnecessary elements or complex workflows.

Another important objective is responsiveness. The interface is designed to respond quickly to user inputs, providing near real-time feedback. This enhances user engagement and creates a smooth interaction experience. The system also focuses on minimizing cognitive load by presenting only essential information and guiding users step by step through the interaction process.

The design also emphasizes consistency. Input fields, buttons, and output displays are arranged in a logical order, ensuring that users can easily understand the flow of interaction. Visual hierarchy is maintained so that important elements such as the prediction result are clearly highlighted.

Interface components

The user interface consists of multiple components that work together to facilitate interaction with the model. The primary input components include a text field for entering the question and four separate fields for entering answer options. These structured inputs ensure that the model receives data in the required format.

A prediction button is provided to initiate the inference process. This button acts as the trigger that sends user inputs to the backend for processing. The use of a single action button simplifies the workflow and makes the system easy to operate.

The output section is designed to display results in a clear and informative manner. It shows the predicted answer along with confidence scores for each option. These scores are typically presented as percentages, allowing users to understand how strongly the model supports each option. This transparency helps users interpret the results more effectively.

In addition to the main components, the interface can include optional features such as example inputs, reset buttons, and tooltips. These elements enhance usability and provide additional guidance to users.

Interaction flow

The interaction flow of the system is designed to be straightforward and efficient. The user begins by entering a question into the input field, followed by the four possible answer options. Once the inputs are provided, the user clicks the prediction button.

Upon clicking the button, the input data is sent to the backend system, where it undergoes preprocessing and tokenization. The processed data is then passed through the trained model, which generates prediction scores for each option. These results are returned to the frontend and displayed in the output section.

The entire process is completed within a few seconds, ensuring that users receive immediate feedback. This real-time interaction is a key feature of the system, as it allows users to experiment with different inputs and observe how the model responds.

Usability features

The frontend incorporates several usability features to enhance the overall user experience. One of the most important features is real-time prediction, which ensures that users receive results quickly without long waiting times. This makes the system more interactive and engaging.

Clear output formatting is another essential feature. The results are displayed in a structured format, with the predicted answer highlighted and confidence scores shown for all options. This helps users easily interpret the results without confusion.

The interface also minimizes input requirements by keeping the number of fields limited to essential elements. This reduces user effort and makes the system more accessible.

Additional usability enhancements include error handling mechanisms that ensure the system behaves correctly even when users provide incomplete or incorrect inputs. For example, the system can prompt users to fill all required fields before proceeding with prediction.

The layout is designed to be clean and organized, with proper spacing and alignment of elements. This improves readability and ensures that users can focus on the task without distractions. The interface can also be made responsive, allowing it to adapt to different screen sizes and devices, including mobile phones and tablets.

Integration with backend

The frontend is tightly integrated with the backend inference system. Gradio acts as a bridge between the user interface and the model, handling input-output communication seamlessly. When a user submits input, Gradio automatically passes the data to the backend function, which processes it and returns the result.

This integration eliminates the need for manual API handling and simplifies the overall system architecture. It also ensures that the frontend remains lightweight while leveraging the full computational power of the backend model.

User experience considerations

From a user experience perspective, the interface is designed to be intuitive and efficient. The simplicity of the design ensures that users can quickly understand how to use the system without requiring instructions. The immediate feedback provided by the model enhances engagement and encourages repeated interaction.

The inclusion of confidence scores adds an element of transparency, allowing users to gauge the reliability of predictions. This is particularly important in mental health applications, where trust and clarity are essential.

Overall, the frontend design successfully combines simplicity, functionality, and responsiveness, creating an interface that makes the MHQA system accessible and practical for real-world use.

The screenshot displays the MHQA web application interface. At the top, the title 'MHQA — Mental Health Question Answering' is shown, followed by technical details: 'Fine-tuned BERT | Accuracy: 79.31% | Domains: Anxiety · Depression · Trauma · OCD'. A disclaimer 'For educational purposes only.' is present. The main input area on the left includes a 'Question' field with a placeholder 'Enter your mental health question...', four 'Option' fields (Option 1 to Option 4), a 'Clear' button, and a prominent orange 'Predict' button. On the right side, the output area contains a 'Predicted Answer' field, a 'Confidence' field, and a section for 'All Option Scores'. Below these is a 'Load Example Topic' dropdown menu currently set to 'Anxiety', and a 'Load Example' button. At the bottom of the interface, a status bar indicates 'Use via API' and 'Built with Gradio'.

BACKEND AND INFERENCE SYSTEM

The backend of the MHQA system plays a crucial role in connecting the user interface with the trained model. It is responsible for processing user inputs, running the model, and returning predictions.

Input processing

When a user submits a query, the backend receives the input and prepares it for processing. The question and answer options are formatted into the required structure and passed through the tokenizer.

Model inference

The tokenized input is fed into the trained BERT model. The model processes the input using its transformer architecture and generates logits for each answer option. These logits represent the model's confidence in each option.

Probability computation

The logits are converted into probabilities using the softmax function. This step ensures that the outputs are normalized and can be interpreted as confidence scores.

Output generation

The option with the highest probability is selected as the predicted answer. The backend also prepares the confidence scores for display, allowing users to see how strongly the model supports each option.

The results are then sent back to the frontend for display.

DEPLOYMENT

DEPLOYMENT

The deployment phase of the MHQA system focuses on transforming the trained BERT-based model into a fully functional and accessible application that can be used in real-world scenarios. Unlike experimental setups confined to local environments, this phase ensures that the model is hosted, served, and interacted with through a scalable and user-friendly interface.

Model saving

After completing the training process in Google Colab, the trained model along with its tokenizer is saved locally. The saving process includes multiple components such as the model weights, configuration files, tokenizer vocabulary, and metadata describing the model setup. These files are stored in a structured directory, typically named something like `mhqa-bert-model`, which contains all necessary artifacts required for inference.

In addition to the standard model files, a configuration JSON file is also created, which stores important information such as the base model used (`bert-base-uncased`), the task type (multiple-choice question answering), the number of choices, maximum sequence length, and achieved accuracy. This structured saving mechanism ensures that the model can be easily reloaded without retraining.

Model hosting

Once the model is saved locally, it is uploaded to Hugging Face Hub. This platform provides a centralized repository for hosting machine learning models and enables easy sharing and reuse. In your implementation, the model is uploaded under a repository similar to:

`bsrisanjana/mhqa-bert-finetuned`

This step involves authenticating with a Hugging Face token, creating a repository, and uploading all model files. The upload process ensures that the model is stored securely and can be accessed globally.

Hosting the model on Hugging Face provides several advantages. It enables version control, easy model sharing, and integration with APIs. Additionally, it allows other developers and researchers to explore and reuse the model, making the system more collaborative and scalable.

Application deployment

The next step involves deploying the model as an interactive web application. This is achieved using Hugging Face Spaces combined with Gradio. Hugging Face Spaces provides an environment for hosting machine learning applications, while Gradio is used to build the user interface.

The Gradio interface is connected directly to the hosted model. When a user enters a question and answer options, the input is sent to the backend, processed by the model, and the prediction is returned to the interface. This integration allows real-time interaction with the model.

The deployment pipeline can be summarized as follows:

Colab trained model → Saved locally → Uploaded to Hugging Face → Integrated with Gradio → Hosted on Hugging Face Spaces

This pipeline ensures a seamless transition from training to deployment.

Accessibility and scalability

One of the key advantages of this deployment approach is accessibility. The application can be accessed through a simple web link without requiring any installation. Users can interact with the system from any device, including laptops, tablets, and smartphones.

The system is also scalable, as Hugging Face Spaces handles the infrastructure required to serve the application. This means that multiple users can interact with the system simultaneously without affecting performance.

Additionally, the deployment supports future enhancements such as API integration, allowing the model to be used in mobile applications, chatbots, or enterprise systems.

RESULTS AND PERFORMANCE ANALYSIS

The performance of the MHQA system is evaluated using the validation dataset during training. The results obtained from the Google Colab training process provide a clear understanding of how well the model performs on unseen data.

Training results

The model was trained for 3 epochs, and the following results were observed:

Epoch 1

Training Loss: 0.5860

Validation Loss: 0.6046

Validation Accuracy: 77.56%

Epoch 2

Training Loss: 0.4981

Validation Loss: 0.6119

Validation Accuracy: 79.27%

Epoch 3

Training Loss: 0.3825

Validation Loss: 0.9244

Validation Accuracy: 79.31%

Final Results

Validation Accuracy: 79.31%

Validation Loss: 0.9244

These results clearly show that the model achieves its best performance around the second and third epochs, with accuracy stabilizing at approximately 79%.

Training behavior

The training process demonstrates a typical deep learning learning curve. Initially, there is a significant improvement in accuracy from the first to the second epoch, indicating that the model is learning meaningful patterns from the data.

From the second to the third epoch, the improvement becomes minimal, and validation loss increases slightly. This suggests the beginning of overfitting, where the model starts to memorize training data instead of generalizing. However, since accuracy remains stable, the model is still considered effective.

The decrease in training loss across epochs confirms that the model is successfully optimizing its parameters.

Dataset impact

A major factor contributing to the improved performance is the use of a large combined dataset (~58,617 samples). Compared to smaller datasets used in Part 1, this significantly enhances the model's ability to generalize.

The dataset provides diverse examples across different mental health domains, allowing the model to learn varied patterns. This results in better performance on unseen questions.

Performance insights

The model performs particularly well on reasoning-based questions, such as prognostic and preventive queries. This is because transformer models like BERT are highly effective at understanding context and relationships between words.

However, the model faces challenges in cases where answer options are very similar. In such scenarios, the probability scores for multiple options are close, leading to occasional misclassification.

Another observation is that the model handles domain-specific terminology effectively, thanks to pretraining on large text corpora.

Inference example (from your notebook)

When tested on a sample validation question, the model correctly predicted the answer with very high confidence:

Predicted Option: 2

Confidence: ~100%

Actual Answer: Option 2

This demonstrates that the model is capable of making highly confident and accurate predictions in certain cases.

The screenshot displays the MHQA interface. On the left, a question is entered: "Over the last two weeks, how often have you been bothered by feeling down, depressed, or hopeless". Below the question are four options: Option 1 (down), Option 2 (bothered), Option 3 (depressed), and Option 4 (sad). A "Predict" button is visible. On the right, the "Predicted Answer" section shows "Option 3: depressed" with a green checkmark. Below this, the "Confidence" is listed as "62.8% confidence". A section titled "All Option Scores" shows a horizontal bar chart with the following scores: Option 1 (5.4%), Option 2 (27.5%), Option 3 (62.8%), and Option 4 (4.4%). At the bottom right, there is a "Load Example Topic" dropdown menu set to "Anxiety" and a "Load Example" button.

Option	Score
Option 1	5.4%
Option 2	27.5%
Option 3	62.8%
Option 4	4.4%

APPLICATIONS

The MHQA system has a wide range of applications, particularly in the fields of mental health, education, healthcare support, and intelligent systems. Due to its ability to understand context and evaluate multiple answer options accurately, the system can be applied in both academic and real-world environments. The combination of domain-specific data and transformer-based learning enables it to provide reliable and structured responses, making it a versatile tool across multiple domains.

One of the primary applications of the system is in educational environments. Students studying psychology, psychiatry, or general health sciences can use the system as an interactive learning tool. By presenting multiple-choice questions along with predictions, the system helps reinforce conceptual understanding. It allows learners to test their knowledge, compare answers, and understand reasoning patterns. Additionally, it can be used in digital classrooms or e-learning platforms to create automated quizzes and assessments related to mental health topics.

Another important application is in mental health awareness and self-learning platforms. Many individuals seek information about symptoms, conditions, and preventive measures but may not always have immediate access to professionals. The MHQA system can serve as an informational assistant that provides structured responses to commonly asked questions. While it does not replace professional diagnosis or treatment, it can help users gain basic awareness about conditions such as anxiety, depression, and trauma. This makes it particularly useful in early-stage information seeking and public awareness initiatives.

The system can also be integrated into chatbot-based applications. AI-driven chatbots are increasingly used in customer support, healthcare, and counseling services. By embedding the MHQA model into such systems, it is possible to provide automated responses to user queries related to mental health. This enables quick and consistent information delivery, especially in situations where human support may not be immediately available. For example, mental health support platforms can use this model to answer frequently asked questions and guide users toward appropriate resources.

In addition to user-facing applications, the system has significant value in research and development. Researchers working in Natural Language Processing and healthcare AI can use the MHQA model as a benchmark for evaluating question answering systems. The model can be used to study performance across different question types, analyze errors, and experiment with new architectures or training strategies. It also provides insights into how transformer models handle domain-specific data.

The system also has potential applications in enterprise and healthcare information systems. In clinical or hospital settings, professionals often need quick access to structured information. The MHQA model can assist in retrieving relevant knowledge from large datasets and presenting it in a concise format. This can improve efficiency and support decision-making processes.

FUTURE WORK

Although the current system demonstrates strong performance and practical usability, there are several opportunities for further improvement and expansion. These enhancements can focus on improving model accuracy, increasing system capabilities, and expanding its applicability to real-world scenarios.

One of the most important areas for future work is the use of more advanced transformer architectures. While BERT provides strong baseline performance, newer models such as RoBERTa, DeBERTa, and LLaMA offer improved contextual understanding and efficiency. These models incorporate architectural improvements and training optimizations that can lead to higher accuracy and better generalization. Fine-tuning such models on the MHQA dataset can significantly enhance system performance.

Another key direction is transitioning from multiple-choice question answering to generative question answering. In the current system, the model selects answers from predefined options. However, generative models can produce descriptive and detailed responses, making the system more flexible and informative.

Explainability is another critical aspect that can be improved. In sensitive domains such as mental health, it is important for users to understand why a particular answer was selected. Techniques such as attention visualization, gradient-based methods, or SHAP values can be used to highlight the parts of the input that influenced the model's decision. This increases transparency and builds trust in the system.

The system can also be extended to support multilingual capabilities. Currently, the model is trained on English data, which limits accessibility for non-English users. By training or fine-tuning multilingual models, the system can be adapted to support multiple languages, making it more inclusive and globally applicable.

Another area of improvement is data augmentation. Increasing the diversity and size of the dataset through techniques such as paraphrasing, back-translation, or synthetic data generation can help improve model robustness. This would allow the model to handle a wider variety of question styles and phrasing patterns.

From a deployment perspective, future work can focus on developing full-scale web and mobile applications. While the current system uses a Gradio interface, a more advanced frontend with improved design, user authentication, and personalized features can enhance user experience.

Real-time integration with external knowledge sources is another potential enhancement. By combining the model with retrieval-based systems, it is possible to provide more accurate and up-to-date information.

Finally, ethical considerations and safety mechanisms can be incorporated into the system. Since mental health is a sensitive domain, it is important to ensure that the system provides responsible and non-harmful responses. Future work can include adding safeguards, disclaimers, and escalation mechanisms to guide users toward professional help when necessary.

FUTURE WORK

In this phase of the project, the MHQA system has been successfully transformed from a trained machine learning model into a complete and deployable application. The implementation demonstrates a full end-to-end pipeline, starting from dataset preparation and preprocessing, followed by model training and evaluation, and finally deployment through a user-friendly interface.

The use of a large-scale dataset, combining MHQA and MHQA-b, plays a significant role in improving the model's performance and generalization capabilities. The transformer-based architecture, specifically BERT, enables the system to understand complex language patterns and perform accurate multiple-choice question answering. The achieved validation accuracy of 79.31% indicates that the model is capable of handling domain-specific queries effectively.

One of the key achievements of this phase is the successful deployment of the model using Hugging Face Hub and Gradio. This allows the system to be accessed through a web interface, making it usable by a wide range of users. The integration of backend inference with a frontend interface demonstrates how machine learning models can be converted into practical applications.

The project also highlights the importance of combining technical performance with usability. A high-performing model alone is not sufficient; it must also be accessible, interpretable, and user-friendly. By incorporating an interactive interface and real-time prediction capabilities, the system becomes more practical and impactful.

Despite its strengths, the system has certain limitations, such as difficulty in distinguishing closely related answer options and limited support for open-ended queries. However, these challenges provide opportunities for future improvements, including the use of advanced models, generative approaches, and explainability techniques.

Overall, this project demonstrates how modern AI technologies can be applied to solve real-world problems in sensitive domains such as mental health. It provides a strong foundation for developing intelligent systems that can support learning, awareness, and decision-making. With further enhancements, the MHQA system has the potential to evolve into a more comprehensive and impactful tool for mental health assistance.

REFERENCES

- [1] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova,
“BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,”
in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [2] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, et al.,
“Transformers: State-of-the-Art Natural Language Processing,”
in *Proceedings of EMNLP: System Demonstrations*, 2020, pp. 38–45.
- [3] Hugging Face,
“Transformers Documentation,”
[Online]. Available: <https://huggingface.co/docs/transformers>
- [4] Hugging Face,
“Hugging Face Hub: Model Hosting Platform,”
[Online]. Available: <https://huggingface.co>
- [5] Gradio Team,
“Gradio: Build Machine Learning Web Apps,”
[Online]. Available: <https://www.gradio.app>
- [6] IndiaAI,
“BharatGen MHQA Dataset,”
[Online]. Available: <https://aikosh.indiaai.gov.in>
- [7] PyTorch Team,
“PyTorch: An Imperative Style, High-Performance Deep Learning Library,”
Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [8] Scikit-learn Developers,
“Scikit-learn: Machine Learning in Python,”
Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [9] Google,
“Google Colaboratory,”
[Online]. Available: <https://colab.research.google.com>
- [10] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, et al.,
“RoBERTa: A Robustly Optimized BERT Pretraining Approach,”
arXiv preprint arXiv:1907.11692, 2019.